



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Flower FRL Benchmark: An Open Testbench for Federated Reinforcement Learning Research

Master Thesis

TODO: Your Name

TODO: Submission Date

Advisors: TODO: Advisor Names

Department of Computer Science, ETH Zürich

Acknowledgments

You can use this section to thank whoever you want to in the context of your thesis. This can of course include family and friends, colleagues, your advisor, and in general anyone that provide some form of support. You can also take the opportunity here to give credit to people who may have enlightened you with discussions, and also certainly your co-authors of an academic paper related to the thesis, should there be one.

Abstract

Federated Reinforcement Learning (FedRL) enables multiple agents to collaboratively learn a shared policy without exchanging raw experience data, making it attractive for privacy-sensitive and distributed control applications. Despite growing theoretical interest, the field lacks a standardised and reproducible testbench that allows researchers to evaluate and compare algorithms under identical conditions.

This thesis presents the *Flower FRL Benchmark*, an open-source framework for federated policy gradient research built on the Flower federated learning platform. The benchmark provides a complete implementation of FedPG-BR — a Byzantine-robust federated policy gradient algorithm — together with SVRPG and a plain averaging baseline. A decorator-based strategy plugin system allows researchers to register a custom aggregation algorithm in a single Python file and have it immediately available for experimentation without modifying any framework code. The system further supports simulation of seven Byzantine attack types, standard discrete and continuous control environments, and a real-time web dashboard for live experiment monitoring.

Through empirical evaluation we characterise the interaction between FedPG-BR’s probabilistic Byzantine filter and policy learning dynamics across environments. We identify and explain a phenomenon in which the filter increasingly suppresses honest agents as training progresses — a consequence of rising gradient variance in improving policies — and discuss its implications for hyperparameter selection and algorithm choice. The benchmark is distributed as a reproducible Docker image and is designed to lower the barrier for academic research in federated reinforcement learning.

Contents

Contents	iii
1 Introduction	1
2 Background and Related Work	4
2.1 Reinforcement Learning	4
2.1.1 Markov Decision Processes	4
2.1.2 Policy Gradient Methods	4
2.1.3 Policy Parameterisation	5
2.2 Federated Learning	5
2.2.1 The Federated Averaging Setting	5
2.2.2 Challenges Specific to Federated RL	5
2.3 Byzantine Robustness	6
2.3.1 The Byzantine Generals Problem	6
2.3.2 Gradient-Based Byzantine Filters	6
2.4 FedPG-BR	6
2.4.1 Byzantine Filtering	6
2.4.2 SCSG Variance Reduction	7
2.5 Related Work	7
2.5.1 Federated Policy Gradient Algorithms	7
2.5.2 Benchmarking in Federated Learning	7
2.5.3 Gymnasium and MuJoCo	8
3 System Design and Implementation	9
3.1 Architecture Overview	9
3.2 Flower Integration	9
3.2.1 Server Strategy	9
3.2.2 Client Application	10
3.2.3 Simulation and Deployment	10
3.3 The Strategy Plugin System	11

3.3.1	Design	11
3.3.2	Auto-Discovery	12
3.3.3	Bundled Strategies	12
3.4	Environments and Configuration	12
3.4.1	Supported Environments	12
3.4.2	Policy Networks	13
3.5	Byzantine Attack Simulation	13
3.6	Web Dashboard	13
3.7	Empirical Observations	14
3.7.1	CartPole-v1	14
3.7.2	LunarLander-v3 and the Filter Degradation Problem .	14
3.7.3	SCSG Ratio Check and Effective Variance Reduction .	15
3.7.4	HalfCheetah-v5	15
4	Conclusion	16
	Bibliography	19
	Dummy Appendix	20

Introduction

Motivation

Modern reinforcement learning systems increasingly operate in settings where experience data is distributed across multiple agents, devices, or geographic locations. Consider a fleet of autonomous vehicles that must collectively learn safe driving policies, or a network of industrial robots sharing a manufacturing task while operating in different physical environments. In such scenarios, centralising all data for training is not only logistically expensive but may be legally or ethically infeasible due to data privacy requirements.

Federated Learning [?] addresses this problem by allowing participants to collaboratively train a shared model while keeping their local data private. Rather than sending raw data to a central server, each participant computes a local model update and transmits only that update. The server aggregates the updates and broadcasts an improved global model. This cycle continues until convergence.

Federated Reinforcement Learning (FedRL) applies this paradigm to the reinforcement learning setting, where each participant is an agent interacting with its own environment. Agents compute local policy gradient estimates from their collected trajectories and send these gradients to a coordination server. The server aggregates the gradients and updates the shared policy. This approach combines the sample-efficiency benefits of sharing experience across agents with the privacy and scalability benefits of federated communication.

The Reproducibility Problem

Despite growing theoretical interest in FedRL, the field suffers from a severe lack of standardised tooling. Published algorithms are typically evaluated in custom experimental setups against custom baselines. Reproducing results from a FedRL paper often requires reimplementing the entire framework from scratch — a process that is time-consuming, error-prone, and frequently leads to unintentional discrepancies. The absence of shared infrastructure makes fair comparison across methods nearly impossible and slows the pace of scientific progress.

This problem is well-recognised in the supervised federated learning community, which has responded with frameworks such as Flower [?], PySyft, and FedML. These tools provide standardised communication protocols, client-server abstractions, and experiment management. However, their support for reinforcement learning workloads — where the fundamental data unit is a variable-length trajectory rather than a fixed-size minibatch — remains limited and largely unevaluated.

The algorithm this work is most directly concerned with is FedPG-BR [?], a Byzantine-robust federated policy gradient method. FedPG-BR extends the basic federated policy gradient approach with a probabilistic filter that identifies and excludes Byzantine agents — workers who send malicious gradient updates to corrupt the shared policy. While the theoretical contributions of FedPG-BR are significant, no open reference implementation existed that would allow other researchers to reproduce its results, compare against it fairly, or build upon it as a baseline.

Our Contribution

This thesis presents the **Flower FRL Benchmark**, an open-source testbench for federated policy gradient research. Built on top of the Flower federated learning framework [?], the benchmark offers:

- A complete implementation of FedPG-BR [?], including its SCSG variance reduction and probabilistic Byzantine filter.
- Two additional aggregation baselines: SVRPG [?] (variance reduction without Byzantine filtering) and GOMDP (plain gradient averaging).
- A *strategy plugin system* allowing researchers to register a custom aggregation algorithm with a single Python decorator and have it immediately available for comparison, without modifying any framework code.
- Simulation of seven Byzantine attack types from the FedPG-BR paper and related literature.

-
- Out-of-the-box support for discrete control environments (CartPole-v1, Acrobot-v1, MountainCar-v0) and continuous control environments (LunarLander, HalfCheetah via MuJoCo [?]).
 - A real-time web dashboard for experiment configuration, live reward monitoring, and per-agent diagnostics.
 - Docker-based deployment for reproducible execution across platforms.

The overarching design goal is that a researcher should be able to implement and test a new FedRL aggregation strategy in a single Python file, with no knowledge of the underlying Flower communication protocol. The benchmark thus serves both as a reproducible reference implementation of FedPG-BR and as an extensible platform for future FedRL research.

Thesis Outline

This thesis is structured as follows:

- **Chapter 2 (Background and Related Work):** We review the foundational concepts in reinforcement learning, policy gradient methods, federated learning, and Byzantine robustness, and describe the FedPG-BR and SVRPG algorithms in detail.
- **Chapter 3 (System Design and Implementation):** We describe the architecture of the Flower FRL Benchmark, including its Flower integration, strategy plugin system, Byzantine attack simulation, and web dashboard.
- **Chapter 4 (Conclusion):** We summarise our contributions, discuss the limitations of the current implementation, and outline directions for future work.

Background and Related Work

2.1 Reinforcement Learning

2.1.1 Markov Decision Processes

We model the agent-environment interaction as a *Markov Decision Process* (MDP), defined by the tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, r, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $P : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \rightarrow [0, 1]$ is the transition kernel, $r : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is the reward function, and $\gamma \in (0, 1)$ is the discount factor.

An agent follows a stochastic policy $\pi_\theta : \mathcal{S} \rightarrow \Delta(\mathcal{A})$, parameterised by $\theta \in \mathbb{R}^d$, where $\Delta(\mathcal{A})$ denotes the probability simplex over $\mathcal{A}$. A *trajectory* $\tau = (s_0, a_0, r_0, s_1, a_1, r_1, \dots, s_T)$ is generated by executing π_θ in the environment. The agent's objective is to find the parameters θ^* that maximise the expected discounted return:

$$J(\theta) = \mathbb{E} \left[\sum_{t=0}^T \gamma^t r(s_t, a_t) \right].$$

2.1.2 Policy Gradient Methods

The *policy gradient theorem* [?] provides an analytical expression for the gradient of $J(\theta)$ with respect to θ :

$$\nabla_\theta J(\theta) = \mathbb{E} \left[\sum_{t=0}^T \nabla_\theta \log \pi_\theta(a_t | s_t) \cdot G_t \right],$$

where $G_t = \sum_{k=t}^T \gamma^{k-t} r_k$ is the discounted return from time step t . This identity allows gradient ascent to be performed using sampled trajectories, without requiring a model of the environment.

The REINFORCE algorithm [?] is the canonical Monte Carlo realisation of this idea. Given a batch of B trajectories sampled from π_θ , the gradient

estimate is:

$$\hat{g}(\theta) = \frac{1}{B} \sum_{i=1}^B \sum_{t=0}^T \nabla_{\theta} \log \pi_{\theta}(a_t^{(i)} | s_t^{(i)}) \cdot G_t^{(i)}.$$

This estimate is unbiased but has high variance, motivating the variance reduction techniques discussed below.

2.1.3 Policy Parameterisation

For discrete action spaces we use a softmax policy:

$$\pi_{\theta}(a | s) = \frac{\exp(f_{\theta}(s, a))}{\sum_{a'} \exp(f_{\theta}(s, a'))},$$

where f_{θ} is a multilayer perceptron (MLP) with parameters θ . For continuous action spaces we use a Gaussian policy whose mean is output by the network and whose variance is a learned parameter.

2.2 Federated Learning

2.2.1 The Federated Averaging Setting

Federated learning [?] considers K workers, each holding a local dataset, that collaboratively train a shared model without sharing their data. In each round, the server broadcasts the current model to all workers. Each worker computes a local gradient update and sends it back to the server. The server aggregates the updates — typically by a weighted average — and applies the result to the global model.

In the FedRL setting, the “local dataset” is replaced by a set of trajectories sampled by the worker from its local environment instance. The locally computed quantity is a policy gradient estimate rather than a supervised loss gradient. The server aggregates these gradient estimates and updates the shared policy parameters.

2.2.2 Challenges Specific to Federated RL

Federated RL inherits the standard challenges of federated learning (communication overhead, statistical heterogeneity across workers) and adds several specific difficulties:

- **Non-stationarity.** The distribution of trajectories depends on the current policy. As the policy improves, the gradient statistics change, requiring aggregation methods that are robust to non-stationary inputs.
- **Variable trajectory length.** Unlike supervised learning minibatches, trajectories have variable length determined by the episode dynamics. This makes communication overhead difficult to bound a priori.

- **High gradient variance.** Policy gradient estimates have notoriously high variance even in the centralised setting. In the federated setting, variance from environment stochasticity compounds with variance from the heterogeneous environments of different workers.

2.3 Byzantine Robustness

2.3.1 The Byzantine Generals Problem

The *Byzantine fault model* [?] considers a system where an unknown subset of f out of K participants may behave arbitrarily — sending corrupted, manipulated, or strategically chosen messages to the aggregation server. In the federated learning context, Byzantine workers send malicious gradient updates designed to corrupt the shared model.

The challenge is that the server cannot distinguish a malicious update from a legitimate one based on its value alone, since honest workers may produce high-variance gradients due to environment stochasticity. Robust aggregation rules must therefore be designed to tolerate a bounded fraction of Byzantine workers while still incorporating useful information from the honest majority.

2.3.2 Gradient-Based Byzantine Filters

Several aggregation rules provide Byzantine robustness by inspecting the statistical properties of the submitted gradients. Coordinate-wise median and trimmed mean are classical examples. More specialised filters exploit the geometry of gradient vectors: if f workers are Byzantine and $K - f$ workers are honest, a filter based on pairwise distances can exclude outliers.

FedPG-BR uses a *probabilistic* filter based on the empirical distribution of gradient norms, parameterised by σ and δ . Crucially, this filter is calibrated to the expected gradient statistics under the current policy, which means its behaviour changes as the policy evolves — an interaction we examine in detail in Chapter 3.

2.4 FedPG-BR

FedPG-BR [?] extends the federated policy gradient framework with two key algorithmic contributions: a Byzantine filter and a variance reduction technique based on SCSG.

2.4.1 Byzantine Filtering

Let $g_1, \dots, g_K \in \mathbb{R}^d$ denote the gradient vectors submitted by the K workers. FedPG-BR computes the empirical mean $\bar{g}$ and uses a probabilistic norm-

based criterion to identify and exclude workers whose gradient deviates excessively from the group:

$$\mathcal{G} = \left\{ i : \|g_i - \bar{g}\| \leq \sigma \cdot \sqrt{d} \right\}.$$

The threshold σ controls the sensitivity of the filter. A small σ filters aggressively; a large σ is permissive. The aggregated gradient is the mean over the good agents $\mathcal{G}$:

$$\mu_t = \frac{1}{|\mathcal{G}|} \sum_{i \in \mathcal{G}} g_i.$$

2.4.2 SCSG Variance Reduction

To reduce the variance of the policy gradient estimate, FedPG-BR uses the *Stochastic Controlled Stochastic Gradient* (SCSG) technique, also explored independently in SVRPG [?]. Starting from the aggregated gradient μ_t at policy θ_t^0 , the server performs a random number $N_t \sim \text{Geom}(p)$ of variance-reduced inner steps. Each inner step uses importance sampling to compute a variance-reduced gradient $v_{t,n}$:

$$v_{t,n} = \hat{g}(\theta_t^n) - \hat{g}_{\text{old}}(\theta_t^n) + \mu_t,$$

where $\hat{g}(\theta_t^n)$ is a mini-batch gradient at the current parameters and $\hat{g}_{\text{old}}(\theta_t^n)$ is a corrected gradient from the snapshot policy θ_t^0 , weighted by the importance ratio:

$$\rho = \frac{\pi_{\theta_t^0}(a | s)}{\pi_{\theta_t^n}(a | s)}.$$

If the importance ratio drifts outside the window $[0.995, 1.005]$, the inner loop terminates to avoid excessive bias from off-policy corrections.

2.5 Related Work

2.5.1 Federated Policy Gradient Algorithms

The federated policy gradient literature has grown rapidly. papini2018svrpg introduced SVRPG, which applies SCSG variance reduction to the centralised policy gradient setting and forms the basis for the variance reduction component in FedPG-BR. Several works have studied convergence guarantees for federated policy gradient under various assumptions on the environment heterogeneity and Byzantine fraction.

2.5.2 Benchmarking in Federated Learning

The supervised federated learning community has developed several benchmarking frameworks, including Flower [?], LEAF, and FedML. These frameworks standardise the communication layer and allow algorithms to be

compared under controlled conditions. The Flower framework in particular has gained wide adoption for its clean Python API and support for both simulation and real distributed deployment.

No analogous benchmark exists for federated reinforcement learning. This gap motivates the present work.

2.5.3 Gymnasium and MuJoCo

Environments are drawn from the Gymnasium library [?], the community-maintained successor to OpenAI Gym. Continuous control experiments use MuJoCo [?], a physics engine widely used in RL research. The specific environments used are CartPole-v1, Acrobot-v1, MountainCar-v0, LunarLander-v3, and HalfCheetah-v5.

System Design and Implementation

3.1 Architecture Overview

The Flower FRL Benchmark is structured as a Python package (`frl_benchmark`) that integrates with the Flower federated learning framework [?]. The system consists of three major components:

1. A **server application** (`server_app.py`) that runs the Flower `ServerApp`, manages the shared policy, and dispatches gradient aggregation to a pluggable strategy.
2. A **client application** (`client_app.py`) that runs the Flower `ClientApp` on each simulated worker, collecting trajectories and computing local policy gradient estimates.
3. A **web dashboard** (`dashboard/`) providing a browser-based interface for experiment configuration and real-time monitoring.

Figure 3.1 illustrates the high-level interaction between these components. In simulation mode, Flower spawns all workers as lightweight virtual nodes within a single process, communicating via in-process message passing. In deployment mode, workers run on separate machines and communicate over gRPC.

3.2 Flower Integration

3.2.1 Server Strategy

The server implements Flower’s `Strategy` abstract class through `FRLStrategy`. The key methods are:

- `configure_fit`: Broadcasts current policy parameters to all workers and specifies the batch size for this round.

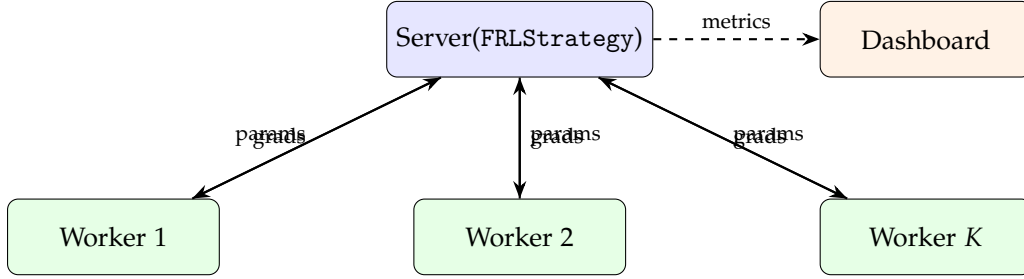


Figure 3.1: High-level architecture of the Flower FRL Benchmark. The server broadcasts policy parameters to K workers, collects gradient estimates, aggregates them via the selected strategy, and pushes metrics to the web dashboard.

- `aggregate_fit`: Collects gradient submissions, removes workers that returned zero examples (skipped rounds), reconstructs gradient tensors, and delegates to the selected aggregation strategy.
- `evaluate`: Evaluates the current policy on 10 episodes using the server’s own environment instance and pushes the result to the dashboard.

The server uses a separate `gymnasium` environment instance for SCSG variance-reduction steps (required by FedPG-BR and SVRPG), keeping the trajectory sampling for gradient computation entirely local to the server.

3.2.2 Client Application

Each worker is represented by a `FRLClient`, which wraps a `Worker` object. In each round, the client:

1. Receives the current global policy parameters from the server.
2. Overwrites its local policy with the received parameters.
3. Samples B trajectories from its local environment, computing the REINFORCE gradient estimate.
4. Returns the gradient as a parameter array together with diagnostic metrics.

Byzantine workers override the gradient computation to substitute a malicious gradient, as described in Section 3.5.

3.2.3 Simulation and Deployment

In simulation mode the entire experiment runs as a single `flower-simulation` process. All workers are spawned as virtual nodes; communication happens in memory. This mode is the primary interface exposed through the web dashboard.

For real distributed deployment, `deploy_server.py` and `deploy_client.py` provide entry points that connect to a Flower superlink. This allows the benchmark to be used across multiple physical machines, reflecting the intended federated setting.

3.3 The Strategy Plugin System

3.3.1 Design

The central design goal of the benchmark is extensibility: a researcher should be able to contribute a new aggregation strategy without modifying any existing code. This is achieved through a *registry pattern* combined with a Python decorator.

Each strategy is a class that inherits from `AggregationStrategy` and implements two methods:

- `aggregate(gradients, batch_size, **kwargs)`: Takes the list of gradient tensors from all active workers and returns an aggregated gradient μ_t and the list of accepted worker indices.
- `server_update(policy, optimizer, theta_t_0, mu_t, config, env)`: Applies the aggregated gradient to the policy, optionally running multiple variance-reduction steps.

Registering a strategy requires only applying the `@register_strategy` decorator:

```
from flr_benchmark.strategies.base import (
    AggregationStrategy, register_strategy, apply_gradient
)

@register_strategy("my-method")
class MyStrategy(AggregationStrategy):
    def aggregate(self, gradients, batch_size, **kwargs):
        mu_t = torch.mean(torch.stack(gradients), dim=0)
        return mu_t, list(range(len(gradients)))

    def server_update(self, policy, optimizer,
                     theta_t_0, mu_t, config, env=None, env_name=""):
        apply_gradient(policy, optimizer, mu_t)
        return 1
```

Once registered, the strategy is immediately selectable from the dashboard's method dropdown without any further changes.

3.3.2 Auto-Discovery

Strategies are discovered at import time via `pkgutil.iter_modules`, which scans the `frl_benchmark/strategies/` directory for Python modules and imports them. This means a researcher can place a new `.py` file in that directory and it will be found automatically on the next run.

3.3.3 Bundled Strategies

Three strategies are provided out of the box:

`fedpg-br` Full FedPG-BR: Byzantine filtering followed by SCSG variance reduction. The Byzantine filter is parameterised by σ and δ ; SCSG uses a geometric distribution over the number of inner steps.

`svrpg` SVRPG [?]: SCSG variance reduction without Byzantine filtering. Accepts all workers unconditionally.

`gomdp` GOMDP: Simple gradient averaging with a single gradient step. No filtering, no variance reduction. Serves as the simplest baseline.

An `example_strategy.py` file demonstrates a complete Trimmed Mean implementation, and `my_strategy.py` provides a template with placeholder methods for users to fill in.

3.4 Environments and Configuration

3.4.1 Supported Environments

The benchmark supports the following environments from Gymnasium:

Environment	Actions	State dim	Notes
CartPole-v1	Discrete	4	Simplest testbed; bounded episodes
Acrobot-v1	Discrete	6	Harder; sparse reward
MountainCar-v0	Discrete	2	Very sparse reward
LunarLander-v3	Discrete	8	Variable episode length
HalfCheetah-v5	Continuous	17	Requires MuJoCo

Table 3.1: Supported Gymnasium environments and their properties.

Each environment has a default configuration stored in `config.py`, including network architecture (hidden units, activation), learning rate, batch size, and filter parameters σ and δ . All parameters can be overridden from the dashboard’s advanced settings panel.

3.4.2 Policy Networks

Discrete-action environments use a softmax MLP policy; continuous-action environments use a Gaussian MLP policy with a learned diagonal covariance. All policies are implemented in `policy.py` and support `get_flat_params` and `set_flat_params` for efficient parameter serialisation over the Flower communication layer.

3.5 Byzantine Attack Simulation

The benchmark simulates seven attack types to enable evaluation of Byzantine robustness:

1. **Random Noise (RN):** Replaces the gradient with a random vector drawn from a standard normal distribution.
2. **Random Action (RA):** The worker takes uniformly random actions during trajectory collection, simulating sensor or hardware failure.
3. **Sign Flipping (SF):** Sends $-2.5 \cdot g$, reversing the gradient direction and amplifying its magnitude.
4. **FedPG Attack:** A sophisticated attack designed to pass the Byzantine filter while still degrading training.
5. **Variance Attack (VA):** Exploits gradient variance by injecting a gradient designed to increase the variance of the aggregated update.
6. **Zero Gradient:** Sends a zero vector, simulating a dropped update without injecting noise.
7. **Reward Flipping:** Negates all rewards during trajectory collection, causing the worker to learn the opposite of the intended behaviour.

Byzantine workers are assigned by index: the first B of K workers are Byzantine. This deterministic assignment ensures reproducibility and matches the convention used in the FedPG-BR paper.

3.6 Web Dashboard

The web dashboard is a Flask [?] application with real-time updates via Flask-SocketIO. It exposes a single experiment page at `/experiment` where the user can:

- Select environment, aggregation method, number of workers, number of Byzantine agents, attack type, and number of rounds.
- Expand an advanced settings panel to override hyperparameters (σ , δ , learning rate, batch size, γ , etc.).

- Start and stop training with immediate effect.
- Monitor a live reward curve updated after every round.
- View per-round logs including the number of good agents retained by the Byzantine filter and the number of SCSG inner steps executed.

When running inside Docker, the dashboard communicates with a separately spawned training container via the Docker API and a shared internal network. When running locally, it launches `flower-simulation` as a subprocess and streams its stdout through the WebSocket to the browser.

3.7 Empirical Observations

3.7.1 CartPole-v1

CartPole-v1 is the most reliable environment in this benchmark. Episodes are capped at 500 steps regardless of policy quality, so per-round wall-clock time remains bounded throughout training. The FedPG-BR Byzantine filter operates stably because gradient statistics do not change dramatically as the policy improves — a solved CartPole policy still collects trajectories of similar length. With $\sigma = 0.06$ and no Byzantine workers, FedPG-BR converges to near-maximum reward within the first 50–100 rounds.

3.7.2 LunarLander-v3 and the Filter Degradation Problem

LunarLander-v3 reveals a fundamental tension between improving policy quality and Byzantine filter stability. Early in training, the lander crashes quickly, producing short episodes and low-variance gradients. The filter retains most workers. As the policy improves, episodes become longer (the lander attempts to land rather than crash immediately), producing higher-variance gradients. The filter, parameterised with $\sigma = 0.07$, begins to classify honest workers as Byzantine — not because they are malicious, but because their gradient norms now fall outside the tight threshold calibrated to early-training statistics.

We observed the number of retained good agents drop from 10 to 5 within 20 rounds on LunarLander even in the absence of any Byzantine workers. This over-filtering effect stalls learning: the aggregated gradient is computed from an increasingly small subset of workers, reducing its quality and slowing convergence.

This observation has a direct practical implication: the σ parameter must be tuned to the expected gradient variance at the intended operating point, not at initialisation. Alternatively, SVRPG (which has no Byzantine filter) avoids this problem entirely at the cost of Byzantine robustness.

3.7.3 SCSG Ratio Check and Effective Variance Reduction

A second observation concerns the SCSG inner loop. The importance ratio check $0.995 \leq \rho \leq 1.005$ is extremely tight. In practice, after even a single gradient step the policy parameters shift enough to push the importance ratio outside this window, causing the loop to terminate with `scsg_steps` = 0 or 1 in most rounds. The effective behaviour of SVRPG in this regime is therefore indistinguishable from GOMDP — plain gradient averaging with a single update step.

This explains why LunarLander performance plateaued at approximately +100 reward with SVRPG: the variance reduction mechanism was not activating, and the system was performing basic policy gradient with no additional signal.

3.7.4 HalfCheetah-v5

The HalfCheetah-v5 environment tests continuous control with a 17-dimensional state space and 6-dimensional action space. MuJoCo episodes are time-limited to 1000 steps regardless of performance, avoiding the episode-length growth problem seen in LunarLander. Initial experiments show FedPG-BR starting at approximately −185 reward and improving steadily over 208 rounds, with the Byzantine filter behaving more stably due to the higher $\sigma = 0.9$ setting appropriate for continuous control.

Conclusion

Summary of Contributions

This thesis presented the **Flower FRL Benchmark**, an open-source testbench for federated policy gradient research. The benchmark addresses a concrete gap in the FedRL ecosystem: while algorithms such as FedPG-BR [?] have established strong theoretical foundations, no shared infrastructure existed for reproducing, comparing, or building upon them.

The main contributions of this work are:

- **A reproducible reference implementation of FedPG-BR.** The benchmark provides a complete implementation of the Byzantine-robust federated policy gradient algorithm, including its probabilistic Byzantine filter and SCSG variance reduction, integrated with the Flower federated learning framework. Experiments can be reproduced from a Docker image with a single command.
- **A strategy plugin system.** Researchers can register a new aggregation strategy using a single Python decorator. The strategy is auto-discovered at runtime and immediately available in the dashboard without any changes to framework code. This lowers the barrier for comparative evaluation of novel FedRL algorithms to the effort of implementing the core aggregation logic.
- **Characterisation of Byzantine filter behaviour under improving policies.** Through empirical evaluation we identified and explained the *filter degradation problem*: FedPG-BR’s probabilistic filter, calibrated to gradient statistics at initialisation, increasingly misidentifies honest workers as Byzantine as the policy improves and gradient variance rises. This phenomenon is an inherent consequence of the non-stationarity of FedRL and has direct implications for the choice of σ and for algorithm selection.

-
- **A real-time web dashboard.** The dashboard enables interactive experiment configuration and live reward monitoring without requiring command-line access, making the benchmark accessible to researchers who are new to the Flower ecosystem.
 - **Byzantine attack simulation.** Seven attack types from the FedPG-BR paper and related literature are implemented, enabling systematic evaluation of algorithm robustness across a range of adversarial scenarios.

Limitations

Several limitations of the current implementation should be noted.

Environments with variable episode length. LunarLander and similar environments where episode length grows with policy quality pose a fundamental challenge for Flower’s simulation mode: as episodes lengthen, rollout time per round increases, which can cause worker timeouts. CartPole-v1 and HalfCheetah-v5 (with their bounded episode lengths) do not suffer from this problem and are more reliable benchmarks for the aggregation algorithms studied here.

SCSG importance ratio threshold. The tight ratio window $[0.995, 1.005]$ used in the SCSG inner loop causes premature termination in practice, rendering the variance reduction ineffective in most rounds. A more adaptive or lenient threshold would likely improve SVRPG’s empirical performance.

Single-node simulation. All workers share the same hardware in simulation mode, which introduces resource contention absent in real distributed deployments. Wall-clock times and convergence profiles may differ when the benchmark is used with actual distributed workers.

Fixed random seeds. The current implementation assigns deterministic per-worker seeds (`client.id`) but does not expose a global experiment seed. Reproducibility across full experiment runs therefore requires attention to the random state of the server-side environment and SCSG sampling.

Future Work

Several directions present themselves as natural extensions.

Adaptive Byzantine filter. The filter degradation problem suggests that σ should adapt over the course of training, tracking the evolving gradient variance rather than being fixed. An exponential moving average of the observed gradient norm distribution could provide a principled adaptive threshold.

Asynchronous FedRL. The current implementation is synchronous: the server waits for all K workers before aggregating. Asynchronous variants, where the server updates on whatever gradients have arrived, are more resilient to slow or failed workers and are an active area of research.

Seed configurability. Adding a global seed parameter to the experiment configuration would allow exact reproduction of individual runs, which is important for debugging and for producing the averaged convergence curves typically reported in academic papers.

Contribution to Flower baselines. The benchmark is designed to be compatible with Flower’s baselines repository. Submitting it as an official Flower baseline would make it discoverable to a wider research community and subject it to the framework’s standard review process.

Additional aggregation strategies. The plugin system makes it straightforward to add further strategies. Coordinate-wise median, geometric median, and Krum are standard Byzantine-robust baselines that would complement the existing FedPG-BR and SVRPG implementations for a more complete comparison.

Bibliography

- [1] Manuel Blum, Robert W Floyd, Vaughan R Pratt, Ronald L Rivest, and Robert E Tarjan. Time bounds for selection. *Journal of Computer and System Sciences*, 7(4):448–461, 1973.
- [2] Robert Bringhurst. *The Elements of Typographic Style*. Hartley & Marks, 1996.

Dummy Appendix

You can defer lengthy calculations that would otherwise only interrupt the flow of your thesis to an appendix.



Eidgenössische Technische Hochschule Zürich
Swiss Federal Institute of Technology Zurich

Declaration of originality

The signed declaration of originality is a component of every written paper or thesis authored during the course of studies. **In consultation with the supervisor**, one of the following two options must be selected:

- ☐ I hereby declare that I authored the work in question independently, i.e. that no one helped me to author it. Suggestions from the supervisor regarding language and content are excepted. I used no generative artificial intelligence technologies¹.
- ☐ I hereby declare that I authored the work in question independently. In doing so I only used the authorised aids, which included suggestions from the supervisor regarding language and content and generative artificial intelligence technologies. The use of the latter and the respective source declarations proceeded in consultation with the supervisor.

Title of paper or thesis:

Authored by:

If the work was compiled in a group, the names of all authors are required.

Last name(s):

First name(s):

With my signature I confirm the following:

- I have adhered to the rules set out in the [Citation Guidelines](#).
- I have documented all methods, data and processes truthfully and fully.
- I have mentioned all persons who were significant facilitators of the work.

I am aware that the work may be screened electronically for originality.

Place, date

Signature(s)

If the work was compiled in a group, the names of all authors are required. Through their signatures they vouch jointly for the entire content of the written work.

¹ For further information please consult the ETH Zurich websites, e.g. <https://ethz.ch/en/the-eth-zurich/education/ai-in-education.html> and <https://library.ethz.ch/en/researching-and-publishing/scientific-writing-at-eth-zurich.html> (subject to change).